

1. Differentiate between List, Set, and Map with suitable example.

List	Set	Map
Ordered Collections of elements	Unordered Collection of elements	Key value pair collection of elements
Duplicates are allowed	No duplicates allowed	Keys are not allowed to be duplicated but values can be duplicated
Allows indexing	Don't allow indexing	Access by key, no indexing.
example: [1, 2, 3, 3]	example: [1, 2, 3]	example = { 1=B, 2=D }
Code: <pre> List<Integer> list = new ArrayList<>(); list.add(1); list.add(2); </pre>	Code: <pre> Set<Integer> set = new HashSet<>(); set.add(1); set.add(2); </pre>	Code: <pre> Map<Integer, String> map = new HashMap<>(); map.put(1, "A"); map.put(2, "B"); </pre>

2. Explain the following list implementations:

- ArrayList
- LinkedList
- Vectors

when would you use each?

1) Arraylist.

- Dynamic Array.
- Fast access ($O(1)$)
- Slow insertion & deletion (shifting needed)
when use: frequent access is required.
- Resizable, most notably Java and C#.

code:

```
import java.util.ArrayList;  
public class Main {  
    public static void main (String[] args) {
```

```
        ArrayList <String> students = new ArrayList<> ();
```

```
        students.add ("Ram");  
        students.add ("Aaryu");  
        students.add ("Nita");  
        System.out.println (students);  
    }  
}
```

2) linked list.

- linear data structure where elements called nodes, are not stored in contiguous memory location.
- Doubly linked list
- fast insertion / deletion
- slow access (traversal needed)
use when: frequent insert / delete.

Code:

```
import java.util.*;  
public class Main {  
    public static void main (String[] args) {  
        linked list <String> list = new linked list <> ();  
        list.add ("A");  
        list.add ("B");  
        list.add ("first"); list.addfirst ("start");  
        System.out.println (list);  
    }  
}
```

3) Vector.

- A data structure representing an ordered, contiguous ~~array~~ ^{sequence} of elements.
- Same as ArrayList but synchronized (thread-safe)
- Slow due to Synchronization

use when: multithreading is needed.

code:

```
import java.util.*;  
Public class Main {  
    public static void main (String[] args) {  
        vector <Integer> vec = new Vector <> ();  
        vec.add (10);  
        vec.add (20);  
        System.out.println (Vec);  
    }  
}
```

3. Why does a set not allow duplicate elements? Explain using hashsets?

⇒ A set doesn't allow duplicate elements because Hashsets using hashing (when we add an element, HashSet calls: element.hashCode(), this generates a number (hashcode) representing the objects) then it finds the memory (called a bucket) and then it checks for duplicates using equals(); if equals() returns true, it means: "This element already exists". So, it does not add it again, if not, it adds.

→ it stores elements based on hashCode().

→ Duplicates are rejected because same hash + equals match.

Example:

```
import java.util.*;  
public class Main {  
    public static void main (String [] args) {  
        HashSet<String> cities = new HashSet<>();  
        cities.add("Kathmandu");  
        cities.add("Biratnagar");  
        cities.add("Kathmandu"); // ignored  
        System.out.println(cities);  
    }  
}
```

4. Explain How a Map stores Data? What is the difference between key and value?

- A map stores data in key value pair (like a dictionary where the word → key and meaning → value).
- key is unique i.e can't be duplicated
 - value can be duplicated.

Example:

```
import java.util.HashMap;
public class map Main {
    public static void main (String [] args) {
        HashMaps < integer, string > students = new HashMap<>();
        student.put (1, "Ram");
        student.put (2, "Sita");
        System.out.println (students);
    }
}
```

5. Difference between

→ ArrayList vs hashset

ArrayList	HashSet.
1. Maintains insertion order	1. No guaranteed order.
2. Duplicates allowed	2. Duplicated not allowed
3. Index access allowed	3. No index access allowed
4. multiple values allowed	4. Only one null value.
5. Performance (Search) is slow ($O(n)$)	5. Performance (Search) is fast. ($O(1)$) avg.
6. Internally Dynamic Array Structure	6. Internally Hash table structure.

→ Hashset vs Hashmap

Hashset

1. Data stored are only elements
2. Duplicates are not allowed
3. Order is not guaranteed
4. One null allowed
5. no index, no key accessed but elements accessed

6. uses hashmap internally

Hash map.

1. Data stored are in key + value pairs.
2. Keys are unique, Values are allowed to be duplicated.
3. Order is guaranteed
4. One null key, multiple null values.
5. accessed via key.

6. uses hashtable internally